

Reporte: Implementación de un Simulador de Estacionamiento mediante POO y MVC

Alumno: Fabricio Reyes Villavicencio

Materia: Paradigmas de la programación

Profesor: José Carlos Gallegos Mariscal

Grupo: 941

Fecha: 03/04/2028

Introducción

El presente documento detalla el proceso de diseño, arquitectura e implementación de un sistema de simulación para la gestión de un estacionamiento. El proyecto surge de la necesidad de aplicar conceptos avanzados de la Programación Orientada a Objetos (POO) en un entorno de desarrollo profesional, utilizando Python como lenguaje base y Flask para la interfaz web.

El enfoque principal se centra en la separación de responsabilidades a través del patrón de diseño Modelo-Vista-Controlador (MVC), asegurando que la lógica del negocio sea totalmente independiente de la interfaz de usuario, ya sea esta una consola de comandos (CLI) o una aplicación web.

Especificación del Problema

De acuerdo con las instrucciones de la Práctica 02, el sistema debe administrar un espacio limitado de cajones de estacionamiento, permitiendo el ingreso de dos tipos de vehículos: automóviles y motocicletas.

Reglas de Negocio Implementadas

- **Gestión de Espacios:** El sistema debe validar que un vehículo solo pueda ocupar un lugar destinado a su tipo.
- **Tarifas Diferenciadas:** Aplicación de un modelo de cobro basado en el tiempo de estancia.
 - Automóviles: \$20.00 MXN por hora.
 - Motocicletas: \$10.00 MXN por hora.
- **Persistencia Temporal:** Gestión de tickets activos en memoria, manteniendo el control de la hora de entrada y la asignación del cajón.

Análisis del Modelo Orientado a Objetos

La arquitectura se fundamenta en los pilares de la POO descritos en el PDF de la práctica:

Abstracción y Encapsulamiento

Se diseñó la clase `ParkingLot` como el controlador central del estado. Esta clase encapsula la lista de objetos `ParkingSpot` y el diccionario de `Ticket`. Los atributos se mantienen privados (mediante la convención de

guion bajo en Python) para evitar manipulaciones externas que corrompan la integridad de los datos, como asignar un vehículo a un lugar ya ocupado.

Herencia y Subtipos

Se implementó una jerarquía de clases para los vehículos:

- **Clase Base:** `Vehicle` (contiene la placa y el tipo de vehículo).
- **Subclases:** `Car` y `Motorcycle`. Esto permite que el sistema utilice el principio de sustitución de Liskov, donde el estacionamiento recibe un objeto `Vehicle` genérico pero se comporta de acuerdo a las propiedades específicas de su subtipo.

Composición

La clase `ParkingLot` no hereda de los lugares de estacionamiento, sino que está **compuesta** por ellos. Esta relación de "tiene un" permite que el estacionamiento sea flexible y pueda escalar en el número de cajones sin alterar su lógica interna.

Polimorfismo mediante Protocolos

Para el cálculo de las tarifas, se utilizó la característica de `typing.Protocol` de Python. Esto define una interfaz de cobro (`RatePolicy`) que permite que el método `calculate` se comporte de manera distinta según el vehículo procesado, eliminando la necesidad de estructuras condicionales extensas y facilitando la adición de nuevas reglas de cobro en el futuro (por ejemplo, tarifas nocturnas o para camiones).

Implementación del Patrón MVC

El Modelo (`models/parking.py`)

Contiene todas las clases de datos y la lógica de validación. No tiene conocimiento de la existencia de Flask ni de la consola. Su única responsabilidad es gestionar los datos del estacionamiento.

La Vista (`templates/`)

Se utilizaron plantillas Jinja2 para renderizar el HTML. Se integró **Bootstrap 5** para garantizar que la interfaz sea visualmente atractiva y responsiva, permitiendo al usuario interactuar de forma intuitiva mediante formularios y tablas de datos.

El Controlador (`app.py` y `cli.py`)

El controlador actúa como intermediario. En la versión web (`app.py`), Flask captura las peticiones HTTP (GET/POST), extrae los datos de los formularios y llama a los métodos correspondientes del modelo `ParkingLot`. Finalmente, redirige al usuario a la vista actualizada.

Desarrollo de las Sesiones

Sesión 1 y 2: Lógica Base y CLI

Se establecieron las clases fundamentales y se probó la lógica mediante una interfaz de línea de comandos. Esta etapa fue crucial para verificar que el cálculo de horas y la liberación de cajones funcionaran correctamente antes de añadir la complejidad de la red.

Sesión 3: Integración Web

Se implementó el servidor Flask. Se crearon rutas para:

- **Dashboard:** Visualización de ingresos totales y ocupación actual.
- **Entry:** Registro de nuevos vehículos.
- **Exit:** Procesamiento de cobro y liberación de espacios.

Conclusiones

El desarrollo de esta práctica permitió comprender la importancia de la arquitectura de software sobre la simple escritura de código. Al separar la lógica del negocio (Modelo) de la interfaz (Vista), se logró crear un sistema robusto, fácil de testear y mantener.

La implementación de polimorfismo y herencia no solo simplificó el código, sino que lo preparó para futuras expansiones. En conclusión, el uso de paradigmas modernos de programación es esencial para el desarrollo de aplicaciones escalables y profesionales en la ingeniería de software actual.